

Asymmetric Key Cryptography

CSCI 4130: Introduction to Cyber Security

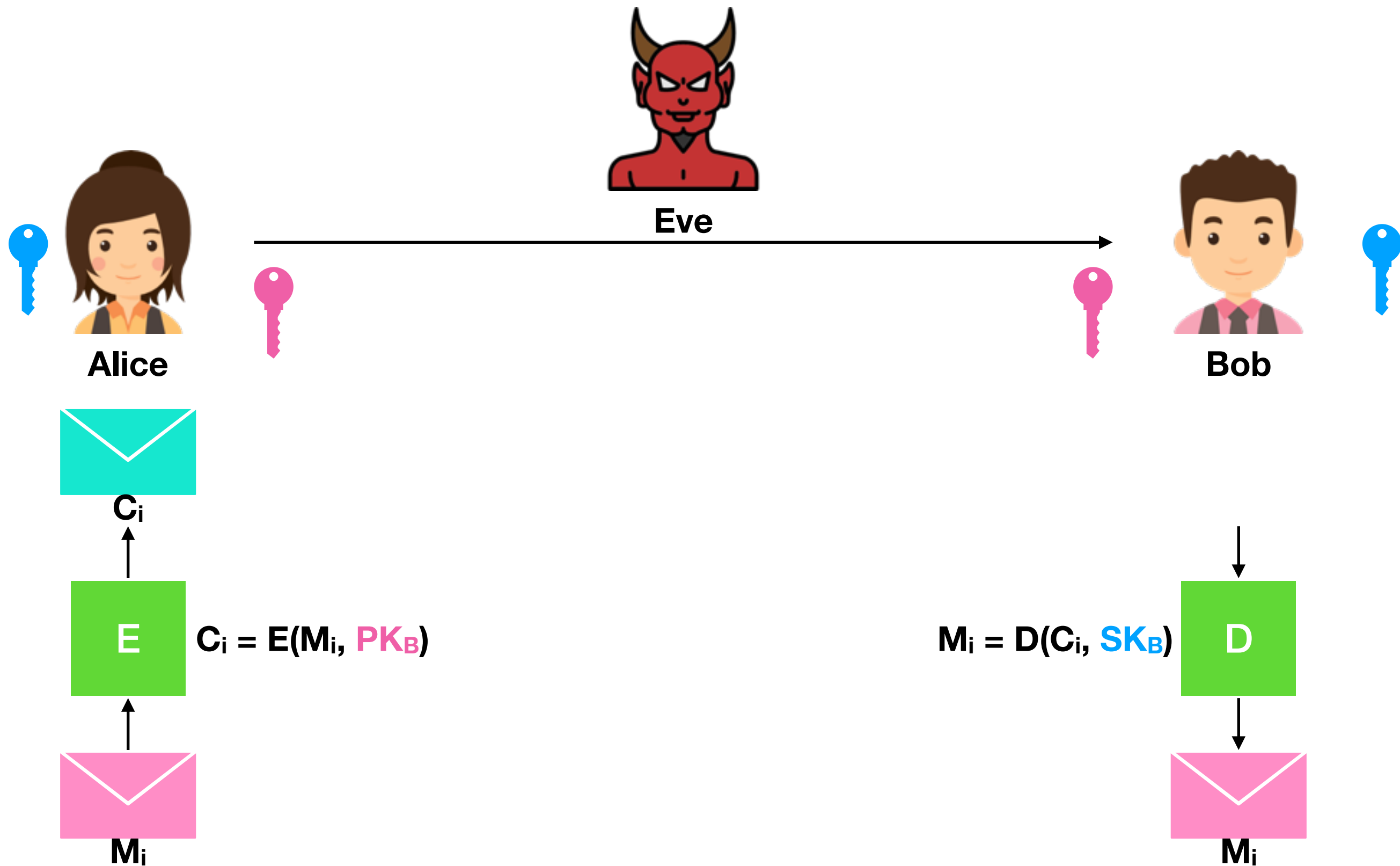
Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Asymmetric Cryptography

- Solve the problem of secret key management/distribution in symmetric encryption
- Each party has a pair of keys {PK, SK}
- The public keys are known
- Asymmetric key encryption
 - Alice encrypts the message with Bob's public key
 - Bob decrypts the message with his private key

Asymmetric Key Encryption



Building Blocks of Asymmetric Key Encryption

- Key Generation Algorithm
 - The generated public key (PK) cannot be used to easily compute the secret key (SK), or vice versa
- Trapdoor permutations
 - One-way permutation without special information
- Encryption Algorithm
 - $C = F(PK, M)$
 - Difficult to inverse without knowing the secret key
- Decryption Algorithm
 - $F^{-1}(SK, C) = M$

One-Way Functions

- A function f such that
 - Given x , it is easy to compute $f(x)$
 - Given y , it is difficult to find any x , s.t., $f(x) = y$
 - The difficulty is measured in a polynomial time computer
- Examples
 - $f(x) = 1$
 - $f(x) = x+1$
 - $f(x) = E_K(x)$, where E_K is a block cipher with a random key K

Euler's Totient Function

- Totient of n
 - $\phi(n)$
 - number of positive integers that are relatively prime to n
 - # of k , $0 < k < n$, s.t. $\gcd(k, n) = 1$
- $\phi(p) = p-1$, if p is a prime number
- $\phi(pq) = \phi(p)\phi(q)$, if p, q are relatively prime
 - The function is multiplicative
- Euler's theorem
 - If a and n co-prime, then $a^{\phi(n)} = 1 \pmod n$

Finding Big Primes Quickly

- 1. Generate a large random number p
- 2. Generate a random integer a : $1 < a < p-1$
- 3. Test $a^{p-1} \bmod p = 1$
 - If not, go to 1, find another p
 - Otherwise, go to 2, generate another a , until we have made enough iterations
 - There exist non-prime numbers for which the equation in Euler's theorem holds
 - They are very rare

RSA Key Generation

- Find **large** primes p, q
- Compute $n = p \cdot q$
- Compute $f(n) = (p-1)(q-1)$
- $f(n)$ is difficult to compute even knowing n
- Choose an e that co-primes with $f(n)$ s.t. $2 < e < f(n)$
- Compute $d = e^{-1} \bmod f(n)$, which is the multiplicative inverse of e , modulo $f(n)$
- PK: $\{n, e\}$
- SK: $\{d\}$

RSA Encryption

- M is an unsigned integer that is smaller than n
- Encryption
 - $C = E(M, PK) = E_{\{n, e\}}(M) = M^e \bmod n$
- Decryption
 - $D(C, SK) = D_{\{d\}}(C) = C^d \bmod n$
 - $= M^{ed} \bmod n$
 - $= (M^{ed-1})M \bmod n$

RSA Encryption (Cont.)

- $e \cdot d = 1 \bmod f(n) \Rightarrow e \cdot d - 1 = k \cdot f(n)$ for some k
 - $(M^{ed-1})M \bmod n = (M^{kf(n)})M \bmod n$
 - $= ((M^{f(n)})^k)M \bmod n$
 - $= (1^k)M \bmod n$ (Euler's Theorem)
 - $= M \bmod n = M$

RSA Encryption (Cont.)

- What if M is no less than n ?
 - Use public-key encryption to exchange a symmetric encryption key
 - Encrypt the message with symmetric encryption
 - This is how public-key encryption commonly used in practice

Diffie-Hellman Key Exchange

- Alice and Bob agree to use a prime p , and a base g
 - g is a primitive root modulo p
 - For every a coprime to p , there is a k , s.t. $g^k = a \pmod{p}$
 - g is a generator of the multiplicative group of integers module p
- Alice chooses a secret a
 - Alice then sends Bob $A = g^a \pmod{p}$
- Bob chooses a secret b
 - Bob then sends Alice $B = g^b \pmod{p}$

Diffie-Hellman Key Exchange (Cont.)

- Alice computes $s = B^a \bmod p$
- Bob computes $s = A^b \bmod p$
- The computation of modular exponentiation (i.e., $g^a \bmod p$) can be done efficiently even for large numbers
- Discrete Logarithm Problem
 - Given only g , p and $A = g^a \bmod p$, it is very difficult to find a , if p is very large (e.g., 600 digits)
- Then, an attacker cannot learn the secret even with the fastest computer

Ephemeral Diffie Hellman

- $s = g^{ab} \bmod p$ is used as the basis for a session key
 - A symmetric key used to protect the communication between Alice and Bob
 - If either a or b is random, s is random
- When Alice and Bob stop the communication, the secrets are discarded
 - This provides forward secrecy
 - Alice and Bob don't retain any information that a later attacker who compromise Alice or Bob's secrets could use to decrypt the messages exchanged with s

MITM Attacks against DH

- The attacker is an active MITM attacker
 - He/She can intercept the message exchanged in the DH protocol
- Eve generates his/her own a' , b'
- Instead of sending $A = g^a \bmod p$, and $B = g^b \bmod p$
- Eve sends $A' = g^{a'} \bmod p$, and $B' = g^{b'} \bmod p$
- Alice will compute $sA' = g^{ab'} \bmod p$
- Bob will compute $sB' = g^{a'b} \bmod p$
- Eve also knows these values!

Digital Signatures

- Goal: Binding a document to the author
 - Attackers can copy Alice's signature from one document to another document
- Solution: Making the signature depend on the document
 - MAC?
 - The signature should be verifiable (publicly)
- $S = \text{sign}(M, SK) = F^{-1}(H(M), SK)$
- $\text{verify}(M, S, PK) = \text{accept}$ if $F(S, PK) = H(M)$

RSA Digital Signatures

- Alice generates a public/private key pair $\{n, e\}$ and $\{d\}$
 - Not recommended to be used for encryption
- Signing a message M
 - $S = \text{sign}_{\{d\}}(M) = \text{RSA}_{\{d\}}(H(M)) = H(M)^d \bmod n$
- Validating the signature
 - $\text{verify}_{\{n, e\}}(M, S) = \text{true}$ iff $H(M) = S^e \bmod n$
- It provides non-repudiation if Alice's public signature key is well-known, because anyone can verify the signature

Asymmetric Key Encryption

- Pros:
 - Key distribution
 - Confidentiality
 - Integrity (weak): Message tampering is detected when decrypting
- Cons:
 - No authentication
 - Repudiation

Asymmetric Key Encryption Summary

- Public Key
 - Something that everyone knows
- Private Key
 - A secret belonging to a specific entity
- Diffie-Hallman
 - A key exchange protocol to share secret
- RSA
 - Provide a ciphertext using the recipient's public key
- RSA Signatures
 - Provide a message that anyone can prove was generated with a private key

A Better Asymmetric Key Encryption

- A better asymmetric key encryption
 - Alice encrypts the message with her private key
 - Alice then encrypts the message with Bob's public key
 - Bob decrypts the message with his private key
 - Bob then decrypts the message with Alice's public key

A Better Asymmetric Key Encryption (Cont.)

- Pros:
 - Confidentiality
 - Integrity
 - Authentication
 - Non-repudiation
- Cons:
 - Vulnerable to MITM attacks

MITM Attacks

- Alice sends her public key to Bob
- Eve intercepts the message and sends Eve's public key to Bob
- Bob sends his public key to Alice
- Eve intercepts the message and sends Eve's public key to Alice
- Alice sends a message encrypted with Eve's public key
- Eve intercepts and decrypts it with her own secret key
- Eve re-encrypts it with Bob's public key and forwards it to Bob

Public-key Certificates

- Bind one's identify with her/his public key
- Such binding is called a certificate
- The certificate is endorsed/guaranteed by a trusted third-party, the Certificate Authority
- The CA has a public key PK_{CA} and a secret SK_{CA}
- The CA can sign a certificate proving the binding of a public key and one identity, given that the CA has verified the identity of the requester
- The CA needs to publish its own certificate (self signed or signed by another trusted CA) to avoid interception

Public-key Certificate Example



***.google.com**

Issued by: Google Internet Authority G3

Expires: Tuesday, December 18, 2018 at 3:43:00 PM Hong Kong Standard Time

✓ This certificate is valid

▼ Details

Subject Name

Country US

State/Province California

Locality Mountain View

Organization Google LLC

Common Name *.google.com

Issuer Name

Country US

Organization Google Trust Services

Common Name Google Internet Authority G3

Serial Number 4848086134840224065

Version 3

Signature Algorithm SHA-256 with RSA Encryption (1.2.840.113549.1.1.11)

Parameters None

Not Valid Before Tuesday, September 25, 2018 at 3:43:00 PM Hong Kong Standard Time

Not Valid After Tuesday, December 18, 2018 at 3:43:00 PM Hong Kong Standard Time

Public Key Info

Algorithm Elliptic Curve Public Key (1.2.840.10045.2.1)

Parameters Elliptic Curve secp256r1 (1.2.840.10045.3.1.7)

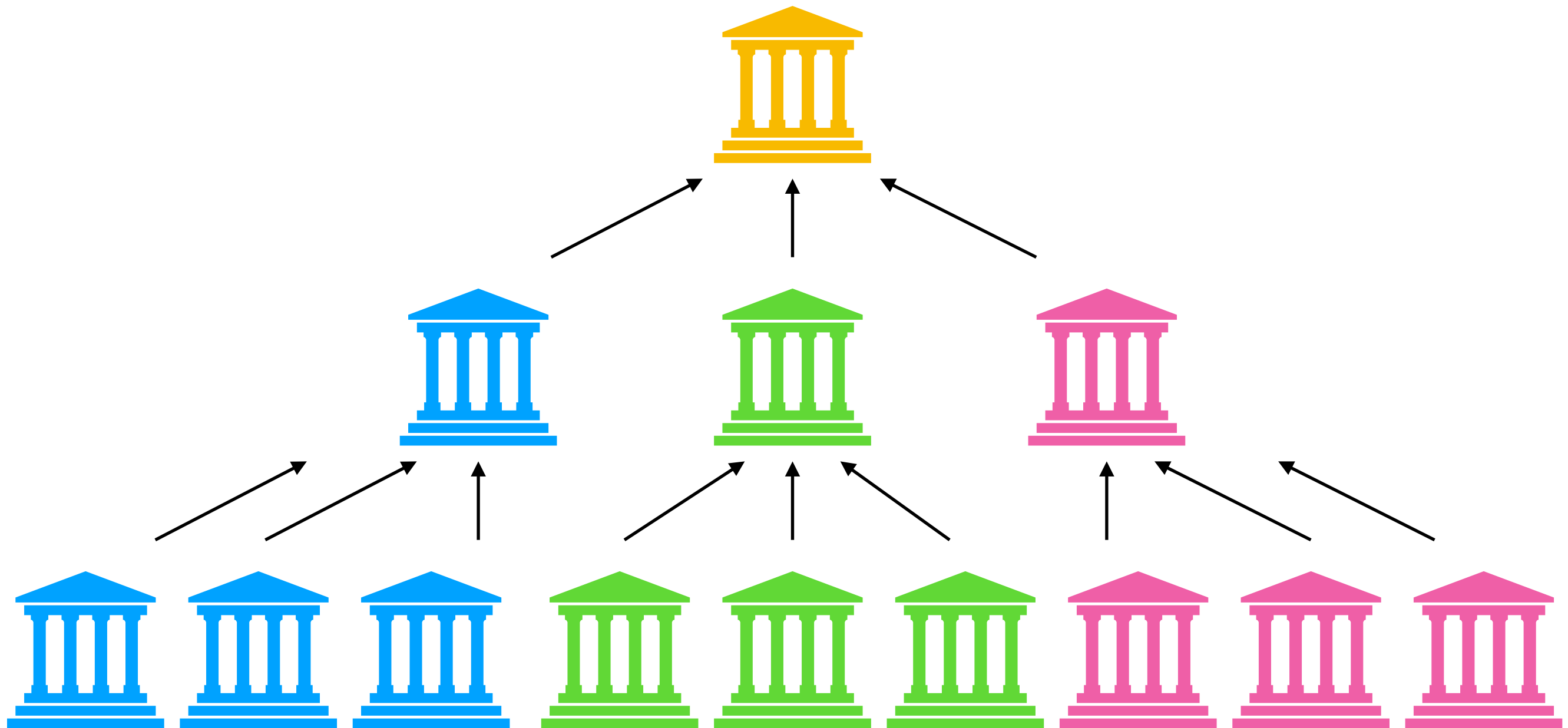
Public Key 65 bytes : 04 A0 42 C6 2B D3 E4 85 ...

Key Size 256 bits

Key Usage Encrypt, Verify, Derive

Signature 256 bytes : 51 3F 52 FC 77 2F B9 A4 ...

Public-Key Infrastructure



Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh, and Giovanni Vigna